

```
import pandas as pd
import re
from collections import defaultdict
from aggregate import file_mapping
from clean_data import paths
from pathlib import Path
import sys

def load_word_set(path):
    return set(
        pd.read_csv(path) ["word"]
        .dropna()
        .astype(str)
        .str.lower()
        .str.strip()
    )

nouns = load_word_set("data/dict/working_set/open_class/nouns.csv")
verbs =
load_word_set("data/dict/working_set/open_class/expanded_verbs.csv")
adjectives =
load_word_set("data/dict/working_set/open_class/adjectives.csv")
adverbs = load_word_set("data/dict/working_set/open_class/adverbs.csv")

auxiliary =
load_word_set("data/dict/working_set/closed_class/auxiliary.csv")
compositions =
load_word_set("data/dict/working_set/closed_class/compositions.csv")
conjunctions =
load_word_set("data/dict/working_set/closed_class/conjunctions.csv")
determiners =
load_word_set("data/dict/working_set/closed_class/determiners.csv")
existential_determiners =
load_word_set("data/dict/working_set/closed_class/existential_determiners.
csv")
modifiers =
load_word_set("data/dict/working_set/closed_class/modifiers.csv")
negative_quantifiers =
load_word_set("data/dict/working_set/closed_class/negative_quantifiers.csv
")
```

```
prepositions =
load_word_set("data/dict/working_set/closed_class/prepositions.csv")
universal_determiners =
load_word_set("data/dict/working_set/closed_class/universal_determiners.csv")
pronouns =
load_word_set("data/dict/working_set/closed_class/pronouns.csv")
possessive_determiners =
load_word_set("data/dict/working_set/closed_class/posessive_det.csv")

COMMON_CONTRACTIONS = {
    "dont": "don't",
    "doesnt": "doesn't",
    "didnt": "didn't",
    "cant": "can't",
    "couldnt": "couldn't",
    "wouldnt": "wouldn't",
    "shouldnt": "shouldn't",
    "wasnt": "wasn't",
    "werent": "weren't",
    "isnt": "isn't",
    "arent": "aren't",
    "hasnt": "hasn't",
    "havent": "haven't",
    "hadnt": "hadn't",
    "wont": "won't",
    "youre": "you're",
    "youve": "you've",
    "youll": "you'll",
    "theyre": "they're",
    "theyve": "they've",
    "itll": "it'll",
    "whats": "what's",
    "thats": "that's",
    "wheres": "where's",
}

VALID_CONTRACTIONS = set(COMMON_CONTRACTIONS.values()) | {
    "i'm", "i've", "i'd", "i'll",
    "it's", "that's", "what's",
```

```

        "you're", "they're", "there's",
        "don't", "doesn't", "didn't",
        "can't", "couldn't", "wouldn't", "shouldn't",
        "wasn't", "weren't", "isn't", "aren't",
        "haven't", "hasn't", "hadn't", "won't",
        "let's", "he's", "she's"
    }

    idx = {
        "Original_Text" : 0,
        "Word_Count" : 1,
        "Char_Count" : 2,
        "Sentiment_Polarity" : 3,
        "Translated_Text" : 4,
        "Source_Id" : 5,
        "Source_Name" : 6
    }

    def tokenize(text):
        if pd.isna(text):
            return []

        text = str(text).lower()

        text = re.sub(r"^[a-zA-Z'\s]", " ", text)

        return text.split()
    print("here")
    #for file in file_mapping:
    #    print(file_mapping.get(file))
    pathing = set()
    for path in paths:
        pathing.add(path[1])
    #for path in pathing:
    #    print(f"pathing: {path}")
    for path in pathing:
        print(f"path: {path}")
        file = "data/dict/working_set/pdfs_as_csvs/" + path + ".csv"
        word_stats = defaultdict(lambda: {
            "frequency": 0,
            "sentiment_sum": 0.0,

```

```

        "review_count": 0
    })
    #name = path[1]
    #print(name)
    #path = Path(path[0])
    #print(f"Name: {str(path[1])}")
    # Load reviews
    text = pd.read_csv(file)

    print("Starting text processing...", flush=True)

    total_reviews = len(text)
    print(f"reviews: {total_reviews}")
#Original_Text,
#Word_Count,
#Char_Count,
#Sentiment_Polarity,
#Translated_Text,
#Source_Id,
#Source_Name

    #incoming data is valid: something here is iterating incorrectly?
    #or maybe it has been incorrectly transformed
    for i, (_, row) in enumerate(text.iterrows(), start=1):
        #print("row(extract_data):")
        #print(_)
        text = _[idx["Original_Text"]]
        #print("Sentiment:")
        #print(_[idx["Sentiment_Polarity"]])
        sentiment = _[idx["Sentiment_Polarity"]]

        sentence = tokenize(text)
        #print(f"words: {sentence}")
        for word in sentence:
            #print(f"word is: {word}")
            #print(f"sentence is: {sentiment}") #sentiment = english...?
This should be a number, not string
            #Check that each row has the correct data
            word_stats[word]["frequency"] += 1

```

```

        word_stats[word]["sentiment_sum"] += sentiment #float to str
error...?

    for word in set(sentence):
        word_stats[word]["review_count"] += 1

    if i % 1000 == 0:
        print(f"Processed {i}/{total_reviews} reviews", flush=True)
        #     word_stats[word]["review_count"] += 1

print("Building word stats...", flush=True)

rows = []
total_words = len(word_stats)

for i, (word, stats) in enumerate(word_stats.items(), start=1):
    frequency = stats["frequency"]

    rows.append({
        "word": word,

        "noun": int(word in nouns),
        "verb": int(word in verbs),
        "adjective": int(word in adjectives),
        "adverb": int(word in adverbs),

        "auxiliary": int(word in auxiliary),
        "compositions": int(word in compositions),
        "conjunctions": int(word in conjunctions),
        "determiners": int(word in determiners),
        "existential_determiners": int(word in
existential_determiners),
        "modifiers": int(word in modifiers),
        "negative_quantifiers": int(word in negative_quantifiers),
        "prepositions": int(word in prepositions),
        "universal_determiners": int(word in universal_determiners),
        "possessive_determiners": int(word in possessive_determiners),
        "pronouns": int(word in pronouns),

```

```

        "frequency": frequency,
        "review_count": stats["review_count"],
        "avg_sentiment": stats["sentiment_sum"] / frequency,
        "length": len(word)
    })

    if i % 1000 == 0:
        print(f"Built {i}/{total_words} word rows", flush=True)

word_df = pd.DataFrame(rows)

POS_COLUMNS = [
    "noun", "verb", "adjective", "adverb",
    "auxiliary", "compositions", "conjunctions", "determiners",
    "existential_determiners", "modifiers", "negative_quantifiers",
    "prepositions", "universal_determiners", "possessive_determiners",
    "pronouns"
]

# Default: keep words unless spellcheck finds a close typo correction
word_df["confident"] = 1
word_df["resolved_word"] = word_df["word"]
# Mark short unknown tokens as NOT confident (garbage...?)
short_garbage_mask = (
    (word_df["word"].str.len() <= 2)
    & (word_df[POS_COLUMNS].sum(axis=1) == 0)
)

word_df.loc[short_garbage_mask, "confident"] = 0
word_df = word_df.sort_values(
    by="frequency",
    ascending=False
)

#word_df.to_csv("data/word_sentiment_stats.csv", index=False)

# -----
# Damerau-Levenshtein typo pass
# -----

```

```

POS_COLUMNS = [
    "noun", "verb", "adjective", "adverb",
    "auxiliary", "compositions", "conjunctions", "determiners",
    "existential_determiners", "modifiers", "negative_quantifiers",
    "prepositions", "universal_determiners", "possessive_determiners",
    "pronouns"
]

known_words = set().union(
    nouns, verbs, adjectives, adverbs,
    auxiliary, compositions, conjunctions, determiners,
    existential_determiners, modifiers, negative_quantifiers,
    prepositions, universal_determiners,
    possessive_determiners, pronouns,
    VALID_CONTRACTIONS
)

# -----
# Faster Damerau-Levenshtein typo pass
# -----

from multiprocessing import Pool, cpu_count
from collections import defaultdict
import time

POS_COLUMNS = [
    "noun", "verb", "adjective", "adverb",
    "auxiliary", "compositions", "conjunctions", "determiners",
    "existential_determiners", "modifiers", "negative_quantifiers",
    "prepositions", "universal_determiners", "possessive_determiners",
    "pronouns"
]

known_words = set().union(
    nouns, verbs, adjectives, adverbs,
    auxiliary, compositions, conjunctions, determiners,
    existential_determiners, modifiers, negative_quantifiers,
    prepositions, universal_determiners,
    possessive_determiners, pronouns
)

```

```
# Group known words by length so we don't scan the whole vocabulary
every time
```

```
known_by_length = defaultdict(list)
for w in known_words:
    known_by_length[len(w)].append(w)
```

```
def damerau_levenshtein(s1, s2):
```

```
    len1 = len(s1)
```

```
    len2 = len(s2)
```

```
    dp = [[0] * (len2 + 1) for _ in range(len1 + 1)]
```

```
    for i in range(len1 + 1):
```

```
        dp[i][0] = i
```

```
    for j in range(len2 + 1):
```

```
        dp[0][j] = j
```

```
    for i in range(1, len1 + 1):
```

```
        for j in range(1, len2 + 1):
```

```
            cost = 0 if s1[i - 1] == s2[j - 1] else 1
```

```
            dp[i][j] = min(
```

```
                dp[i - 1][j] + 1,
```

```
                dp[i][j - 1] + 1,
```

```
                dp[i - 1][j - 1] + cost
```

```
            )
```

```
            if (
```

```
                i > 1 and j > 1
```

```
                and s1[i - 1] == s2[j - 2]
```

```
                and s1[i - 2] == s2[j - 1]
```

```
            ):
```

```
                dp[i][j] = min(dp[i][j], dp[i - 2][j - 2] + 1)
```

```
    return dp[len1][len2]
```

```
def max_allowed_distance(word):
```



```

    if len(word) <= 1:
        return 0
    elif len(word) <= 6:
        return 1
    else:
        return 2

def find_best_match_for_row(row_dict):
    word = row_dict["word"]
    max_dist = max_allowed_distance(word)

    if max_dist == 0:
        return None

    candidates = []

    for length in range(len(word) - max_dist, len(word) + max_dist +
1):
        candidates.extend(known_by_length.get(length, []))

    best_word = None
    best_dist = float("inf")

    for candidate in candidates:
        dist = damerau_levenshtein(word, candidate)

        if dist < best_dist:
            best_dist = dist
            best_word = candidate

        # Can't do better than exact distance 1 for typo correction
here
        if best_dist == 1:
            break

    if best_word is not None and best_dist <= max_dist:
        return {
            "unknown_word": word,
            "suggested_word": best_word,

```

```

        "distance": best_dist,
        "frequency": row_dict["frequency"],
        "avg_sentiment": row_dict["avg_sentiment"]
    }

    return None

print("Finding unrecognized words...", flush=True)
unrecognized = word_df[word_df[POS_COLUMNS].sum(axis=1) == 0].copy()

unrecognized = unrecognized[unrecognized["frequency"] >= 3]

def resolve_contraction(word):
    return COMMON_CONTRACTIONS.get(word, word)

unrecognized["resolved_word"] =
unrecognized["word"].apply(resolve_contraction)

# Anything that changed is probably not a POS-table gap.
# Drop it from the unknown-word review list.
true_unknowns = unrecognized[
    unrecognized["resolved_word"] == unrecognized["word"]
].copy()

rows_to_check = true_unknowns[["word", "frequency",
"avg_sentiment"]].to_dict("records")
print(f"Unrecognized words to check: {len(rows_to_check)}", flush=True)
print(f"Known vocabulary size: {len(known_words)}", flush=True)
print(f"CPU cores available: {cpu_count()}", flush=True)

# Apply contraction resolutions to main dataframe
for _, row in unrecognized.iterrows():
    original = row["word"]
    resolved = row["resolved_word"]

    if original != resolved:
        word_df.loc[word_df["word"] == original, "resolved_word"] =
resolved
        word_df.loc[word_df["word"] == original, "confident"] = 1
    start = time.time()

```

```

suggestions = []

workers = max(1, cpu_count() - 1)
if __name__ == "__main__":
    with Pool(processes=workers) as pool:
        for i, result in
enumerate(pool.imap_unordered(find_best_match_for_row, rows_to_check,
chunksize=100), start=1):
            if result is not None:
                suggestions.append(result)

            if i % 1000 == 0:
                elapsed = time.time() - start
                print(f"Checked {i}/{len(rows_to_check)} words |
Suggestions: {len(suggestions)} | Elapsed: {elapsed:.1f}s")

suggestions_df = pd.DataFrame(suggestions)
if not suggestions_df.empty:

    # FIRST create changed_suggestions
    changed_suggestions = suggestions_df[
        suggestions_df["unknown_word"] !=
suggestions_df["suggested_word"]
    ].copy()

    # THEN filter contractions
    changed_suggestions = changed_suggestions[
        ~changed_suggestions["unknown_word"].isin(VALID_CONTRACTIONS)
    ]

    # Now continue normally
    changed_suggestions["unknown_len"] =
changed_suggestions["unknown_word"].str.len()
    changed_suggestions["suggested_len"] =
changed_suggestions["suggested_word"].str.len()
    changed_suggestions["length_diff"] = (
        changed_suggestions["unknown_len"] -
changed_suggestions["suggested_len"]
    ).abs()

```

```

        changed_suggestions["change_ratio"] = (
            changed_suggestions["distance"] /
changed_suggestions["unknown_len"]
        )

        distance_1_typo = (
            (changed_suggestions["distance"] == 1)
            & (changed_suggestions["unknown_len"] >= 4)
        )

        distance_2_typo = (
            (changed_suggestions["distance"] == 2)
            & (changed_suggestions["unknown_len"] >= 8)
            & (changed_suggestions["length_diff"] <= 1)
            & (changed_suggestions["change_ratio"] <= 0.25)
        )

        plural_to_singular = (
            changed_suggestions["unknown_word"].str.endswith("s")
            & (
                changed_suggestions["unknown_word"].str[:-1]
                == changed_suggestions["suggested_word"]
            )
        )

        min_length_filter = changed_suggestions["unknown_word"].str.len()
>= 4

        reliable_typos = changed_suggestions[
            (distance_1_typo | distance_2_typo)
            & ~plural_to_singular
            & min_length_filter
        ].copy()

        low_confidence_words = set(reliable_typos["unknown_word"])

        suggestion_map = dict(zip(
            reliable_typos["unknown_word"],
            reliable_typos["suggested_word"]

```

```

    ))

    word_df.loc[
        word_df["word"].isin(low_confidence_words),
        "confident"
    ] = 0

    word_df.loc[
        word_df["word"].isin(low_confidence_words),
        "resolved_word"
    ] = word_df["word"].map(suggestion_map)

    print(f"Changed suggestions: {len(changed_suggestions)}",
flush=True)

    print(f"Reliable typos marked confident=0:
{len(low_confidence_words)}", flush=True)

    # Save final updated stats file AFTER confidence changes
    out = "data/dict/working_set/stats/" + path + "_stats.csv"
    word_df.to_csv(out, index=False)
    print("Saved updated stats", flush=True)
    true_unknowns = true_unknowns.sort_values(
        by="frequency",
        ascending=False
    )

    true_unknowns[["word", "frequency", "review_count",
"avg_sentiment"]].to_csv(
        "data/likely_pos_gaps.csv",
        index=False
    )

    print(f"Saved {len(true_unknowns)} likely POS gaps to
data/likely_pos_gaps.csv", flush=True)

    elapsed = time.time() - start
    print(f"Done. Saved {len(suggestions)} suggestions to
data/typo_suggestions.csv")
    print(f"Total typo pass time: {elapsed:.1f}s")

```

